

Proposed Directory Structure and System Overview

This section describes the recommended directory structure and how the re-structured DAFCOM ML pipeline would operate using modular, class-based components.

Directory Structure

```
DAFCOM/
|-- config.yaml          # Master configuration file
|-- main.py              # Pipeline entry point
|-- Refactoring_Plan.md  # Refactoring plan and documentation
|-- README.md            # Project overview
|
|-- src/
|   |-- forecast_mode/
|   |   |-- dataloader.py    # DataLoader class for all data IO
|   |   |-- preprocessor.py  # Preprocessor class for data cleaning/feature engineering
|   |   |-- trainer.py       # ModelTrainer class for ML training
|   |   |-- evaluator.py     # Evaluator class for metrics and validation
|   |   |-- visualization.py # Visualization class for plotting and reporting
|   |   |-- config.py        # Config class for YAML config management
|   |   |-- logger.py        # Logger class for standardized logging
|   |   |-- models/
|   |   |   |-- xgb_model.py # XGBModel class
|   |   |   |-- lstm_model.py # LSTMModel class
|   |   |-- utils.py         # Utility functions
|
|-- tests/                # Unit tests for each class/module
|   |-- test_dataloader.py
|   |-- test_preprocessor.py
|   |-- test_trainer.py
|   |-- test_visualization.py
|   |-- ...
|
|-- docs/                 # Documentation
|   |-- ...
|
|-- .github/
|   |-- workflows/
|   |   |-- ci.yml          # CI/CD configuration
|   |-- issue_templates/
|   |   |-- bug_report.md   # Bug report template
```

```
|      |-- feature_request.md # Feature request template
|      |-- pull_request_templates/
|      |-- default.md         # Default PR template
|-- pyproject.toml            # Project metadata and dependencies
|-- .gitignore                # Git ignore file
```

System Workflow

1. **Configuration**
 - `main.py` loads settings from `config.yaml` using the `Config` class.
2. **Data Loading**
 - The `DataLoader` class reads input data (CSV, NetCDF, etc.) using `pandas/xarray/dask` as appropriate.
3. **Preprocessing**
 - The `Preprocessor` class cleans and transforms the data, supporting parallelism with `dask`.
4. **Model Training**
 - The `ModelTrainer` class coordinates training using model classes (e.g., `XGBModel`, `LSTMModel`).
 - Models are modular and can be swapped/configured via `config.yaml`.
5. **Evaluation**
 - The `Evaluator` class computes metrics and validation results.
6. **Visualization**
 - The `Visualization` class handles all plotting and reporting (e.g., `matplotlib`, `seaborn`, `plotly`), keeping visualization logic modular and reusable.
7. **Logging**
 - The `Logger` class standardizes all output and status messages.
8. **Testing & CI**
 - All classes have corresponding unit tests in `tests/`.
 -

CI/CD runs tests automatically on push.

Visualization Class Example

```
import matplotlib.pyplot as plt

class Visualization:
    def __init__(self, config):
        self.config = config

    def plot_timeseries(self, df, x_col, y_col, title=None):
        plt.figure(figsize=(10, 5))
```

```
plt.plot(df[x_col], df[y_col])
plt.title(title or f"{y_col} over {x_col}")
plt.xlabel(x_col)
plt.ylabel(y_col)
plt.show()
```

Rationale

Visualization code is often scattered across scripts, making it hard to maintain or reuse. A dedicated Visualization class centralizes all plotting and reporting, encourages code reuse, and makes it easy to update or extend visualization capabilities (e.g., switch from matplotlib to plotly) without touching core pipeline logic.

This structure ensures modularity, scalability, and maintainability, and supports incremental development and testing. Each class is responsible for a single aspect of the pipeline, and configuration is centralized for reproducibility and flexibility.